LiquidHaskell: Experience with Refinement Types in the Real World

Niki Vazou, Eric L. Seidel, Ranjit Jhala

Szabó Márk

Haskell's Type System

Haskell is a **statically typed** language.

- Every expression has a type.
- Types are checked before the program runs.
- Types are often inferred automatically.

Basic Types

Some common built-in types:

```
Int        -- integer
Float      -- floating point number
Double     -- floating point number
Num        -- [Int, Float, Double, ...]
Bool       -- True or False
Char       -- single character
String     -- text
```

Type Annotations

The operator :: means “has type”.

```
age :: Int
age = 23

name :: String
name = "Mark"

active :: Bool
active = True
```

Function Types

The arrow (→) separates input and output types.

```
add :: Int -> Int -> Int
add x y = x + y
```

- First Int: first argument
- Second Int: second argument
- Last Int: result

Equivalent in a C-like language:

```
int add(int x, int y) { return x + y; }
```

Type Inference

Haskell can infer many types automatically.

```
1 age = 23
2 double x = x * 2
```

In GHCi we can inspect inferred types:

```
1 :t True
2 True :: Bool
3
4 :t "hello"
5 "hello" :: String
6
7 :t double
8 double :: Num a => a -> a
```

Limits of The Type System

We can’t define constraints for the types of a function like:

- Conditions about the value: `x > 0`
- Relationships between values: `a > b`

To check for positive we need to check inside the function:

```haskell
sqrtRoot :: Double -> Double
sqrtRoot x
  | x > 0 = sqrt x
  | otherwise = error "x must be greater than 0"
```

LiquidHaskell

Many of these properties and relationships can be verified using a combination of **Refinement Types** and **external SMT solvers**.

This paper introduces and experiments with **LiquidHaskell** a refinement typechecker used to verify more than **10.000** lines of Haskell code across popular libraries.

Including: *bytestring*, *text*, *vector-algorithms* and *xmonad*.

Refinement Types

Refinement types:

- contain a **predicate** which is assumed to be true for any element of the refined type.
- can express **preconditions** when used as function arguments or **postconditions** when used as return types.

Examples:

positive natural numbers

$x : \{n \in \mathbb{N} \mid n > 0\}$

even natural numbers

$x : \{n \in \mathbb{N} \mid n \bmod 2 = 0\}$

result is a positive number

$f : \mathbb{N} \to \{y \in \mathbb{N} \mid y > 0\}$

result > input

$f(x : \mathbb{N}) : \mathbb{N} \to \{y \in \mathbb{N} \mid y > x\}$

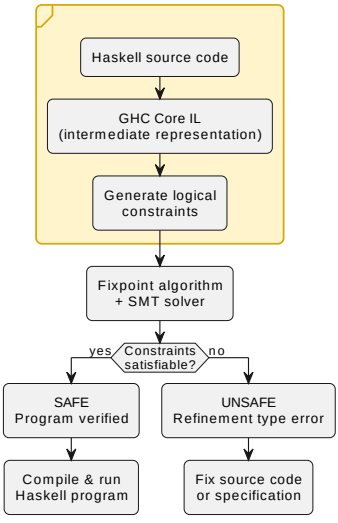
LiquidHaskell Architecture

Core

- Core is a simpler intermediate representation (GHC translates Haskell source to it)
- Source locations are preserved for error reporting

Constraints

- Refinement types are translated into logical constraints
- These constraints describe the properties that must hold



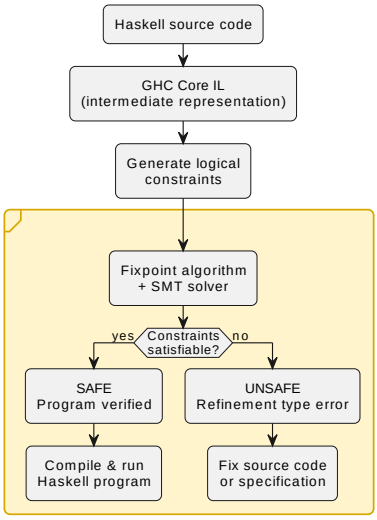
LiquidHaskell Architecture

Solution

- A fixpoint algorithm processes the constraints
- Any *SMT-LIB2* compatible solver checks whether the constraints can be satisfied (eg.: *Z3*, *CVC4*, …)

Result

- **SAFE** or **UNSAFE**: based on the verification of the refinement properties
- Errors are mapped back to their source-code locations



LiquidHaskell

LiquidHaskell is an **optional** type checker for Haskell. Refinement types are written inside **comments**, so they are optional and normal Haskell compilers can ignore them:

```
1 {-@ ... @-}
```

Execution

LiquidHaskell performs static verification before the Haskell program is executed. It does not become part of the executable or perform any checks during runtime.

LiquidHaskell Basics

```
1  {-@ type Natural = {v:Int | v >= 0} @-}
2  {-@ type Pos = {v:Int | v > 0} @-}
3
4  {-@ increment :: Natural -> Pos @-}
5  increment :: Int -> Int
6  increment x = x + 1
7
8  good1 = increment 0
9  good2 = increment 4
10
11 -- verification will fail here stating: Type Mismatch
12 bad = increment (-1)
```

LiquidHaskell Basics

```haskell
{-@ type Natural = {v:Int | v >= 0} @-}
{-@ type Pos = {v:Int | v > 0} @-}

{-@ divide :: n:Natural -> d:Pos -> {v:Natural | v <= n} @-}
divide :: Int -> Int -> Int
divide x y = x `div` y

good = divide 5 2

-- With LH -> Type Mismatch before executing the code
-- Without LH -> runtime error: divide by zero
bad = divide 5 0
```